

Bayesian Optimization for Efficient Hyperparameter Tuning in Fraud Detection Applications

Date: April 29, 2026

Table of Contents

1. Problem Statement and Motivation
2. Methodology: Bayesian Optimization
3. Experimental Design
4. Simulation Study: Proof-of-Concept
5. Analyses and Visualizations
6. Project Development Summary
7. Why This Project is Meaningful
8. Simulation Results Summary
9. Business Impact and Deployment Perspective

1. Problem Statement and Motivation

Credit card fraud detection is a critical application in financial services where model performance directly impacts risk management and customer trust. The IEEE-CIS Fraud Detection dataset (from Vesta Corporation) contains real-world transaction data with 590,571 anonymized transactions, where only ~3.5% are fraudulent. This competition-grade dataset presents multiple challenges:

Data Characteristics:

- **Severe Class Imbalance:** Only ~20,663 fraud cases vs ~570k legitimate transactions
- **High Dimensionality:** 871 total columns after merging transaction and identity tables
- **Mixed Feature Types:**
 - 339 PCA-anonymized numeric features (V1 to V339, no interpretability)

- 14 aggregated count features (C1 to C14, transaction history summaries)
- 6 card-related categorical features (card1 to card6)
- 27 identity match features (id_12 to id_38, sparse)
- Device info and email domain features
- **Missing Data:** Identity table (~144k rows) does not cover all transactions; many features sparse after merge
- **Temporal Aspect:** TransactionDT encodes temporal ordering (relative timestamps)

Why Hyperparameter Tuning Matters Here:

1. **Class Imbalance Penalty:** LightGBM's `scale_pos_weight` parameter is critical for fraud detection. A poorly tuned penalty can either:
 - Over-penalize fraud detection (high false negatives, costly missed fraud)
 - Over-penalize legitimate transactions (high false positives, poor customer experience)
2. **Feature Regularization:** With 871 features and only ~20k fraud examples, regularization (L1/L2) is essential to prevent overfitting
3. **Computational Cost:** Each hyperparameter configuration requires stratified 5-fold CV on 590k+ records (~3 million training samples across folds)
4. **Metric Selection:** Average Precision (AP) is the implemented evaluation metric for the fraud detection experiments because it is more appropriate than AUC-ROC for imbalanced data. AUC-ROC remains a useful secondary perspective for future work.

Research Questions:

- Can Bayesian Optimization with Gaussian Process surrogate and adaptive acquisition functions (EI vs LCB) efficiently discover hyperparameters and outperform random search and grid search on the IEEE-CIS fraud detection dataset?
 - Which hyperparameters most influence fraud detection performance? (via GP kernel lengthscale analysis)
 - How does optimizing `scale_pos_weight` affect the fraud model's precision-recall tradeoff?
-

2. Methodology: Bayesian Optimization

2.1 Gaussian Process Surrogate Model

A Gaussian Process is a non-parametric Bayesian model that places a prior over functions. After observing n evaluations $\mathcal{D}_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the GP posterior at a new point $\mathbf{x}$ has:

Posterior Mean:

$$\mu_n(\mathbf{x}) = \mathbf{k}(\mathbf{x}, \mathbf{X})[K(\mathbf{X}, \mathbf{X}) + \sigma^2 I]^{-1} \mathbf{y}$$

Posterior Variance:

$$\sigma_n^2(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}) - \mathbf{k}(\mathbf{x}, \mathbf{X})[K(\mathbf{X}, \mathbf{X}) + \sigma^2 I]^{-1} \mathbf{k}(\mathbf{X}, \mathbf{x})$$

where:

- $\mathbf{k}(\mathbf{x}, \mathbf{X})$ is the vector of kernel covariances between $\mathbf{x}$ and training inputs
- $K(\mathbf{X}, \mathbf{X})$ is the kernel covariance matrix
- σ^2 is the noise variance
- $\mathbf{y}$ are observed objective values

Kernel Choice: We use the Matern kernel with $\nu = 2.5$, which provides a good balance between smoothness and flexibility for hyperparameter landscapes.

Toy Example: A small 1D toy example demonstrates the GP posterior update and EI calculation step by step, so the Bayesian machinery is shown with concrete numbers before the full high-dimensional experiment.

2.2 Expected Improvement (EI) Acquisition Function

The Expected Improvement acquisition function balances exploration (searching uncertain regions) and exploitation (improving on the current best):

$$\text{EI}(\mathbf{x}) = \mathbb{E} [\max(f(\mathbf{x}) - f^* - \xi, 0)] = (\mu(\mathbf{x}) - f^* - \xi)\Phi(Z) + \sigma(\mathbf{x})\phi(Z)$$

where:

- $f^* = \max_i y_i$ is the current best observation
- $Z = \frac{\mu(\mathbf{x}) - f^* - \xi}{\sigma(\mathbf{x})}$
- $\Phi(\cdot)$ is the standard normal CDF
- $\phi(\cdot)$ is the standard normal PDF

- $\xi \geq 0$ is the exploration bonus (trade-off parameter)

Interpretation:

- **First term:** Exploitation, how much improvement is expected on average above current best
- **Second term:** Exploration, expected gain from high-uncertainty regions, weighted by probability of improvement

The EI acquisition function is grounded in Bayesian decision theory: it selects the point that maximizes expected utility (improvement) under the posterior distribution over functions.

2.3 Upper Confidence Bound (UCB) Acquisition Function

UCB is a simpler acquisition function that combines mean and uncertainty:

$$\text{UCB}(\mathbf{x}) = \mu(\mathbf{x}) + \beta \cdot \sigma(\mathbf{x})$$

where $\beta \geq 0$ controls the explore/exploit trade-off:

- **Low β (e.g., 0.5):** Exploitation, favor regions with high predicted mean
- **High β (e.g., 3.0):** Exploration, favor regions with high uncertainty

Empirical Study: We test $\beta \in \{0.5, 1.5, 3.0\}$ to observe how the parameter influences search behavior and final performance.

3. Experimental Design

3.1 Data and Preprocessing

Dataset: IEEE-CIS Fraud Detection (Vesta Corporation, original dataset source). Dataset files are not included in this repository due to size.

- **Source:** Provided by Vesta Corporation (original dataset distributor).
- **Training Size:** ~590,571 transactions, 871 total columns
- **Class Distribution:** ~3.5% fraud (~20,663 positive), 96.5% legitimate (~570k negative)
- **Files:**
 - `train_transaction.csv` : Main transaction table (~590k rows, 394 columns including target)
 - `train_identity.csv` : Supplementary identity/device info (~144k rows, 41 columns)

- Join key: `TransactionID` (left join; not all transactions have identity data)

Feature Categories:

Numeric Features (Transaction):

- `TransactionDT` : Timedelta from reference datetime (proxy for temporal ordering)
- `TransactionAmt` : Transaction amount in USD
- `V1` to `V339` : 339 PCA-transformed features from Vesta (anonymized, no interpretation needed)
- `C1` to `C14` : 14 count features (frequency of various transaction attributes)

Categorical Features (Transaction):

- `ProductCD` : Product code
- `card1` to `card6` : Card identifiers and related info
- `addr1`, `addr2` : Address information
- `P_emaildomain`, `R_emaildomain` : Payer/Recipient email domains
- `M1` to `M9` : Match features (binary/categorical)

Categorical Features (Identity):

- `DeviceType` : Mobile/desktop/etc.
- `DeviceInfo` : Device identifier string
- `id_12` to `id_38` : 27 additional identity match features

Target:

- `isFraud` : Binary (0 = legitimate, 1 = fraudulent)

Preprocessing Pipeline:

1. Merge `train_transaction.csv` and `train_identity.csv` on `TransactionID` (left join).
After merge, dataset will have ~590k rows with some columns from identity being sparse
2. Drop columns with >80% missingness. Many identity columns may be sparse; remove columns where >80% are NaN
3. Median imputation for remaining numeric features. Handles missing values in `V1` to `V339`, `C1` to `C14`, and other numeric columns
4. Mode imputation for categorical features. Fill missing categorical values with most frequent category per column
5. Label encode categorical columns. Convert `ProductCD`, `DeviceType`, email domains, etc. to integers (LightGBM-compatible), preserving ordinality for ordinal categories if any

6. No scaling applied. LightGBM is tree-based and invariant to monotonic transformations; scaling unnecessary

Data Quality Checks:

- Confirm positive class rate remains ~3.5% after merge and imputation
- Verify no leakage from test set information
- Ensure TransactionID is unique and no duplicates introduced during merge

Validation: Stratified 5-fold cross-validation (stratification essential at 3.5% positive rate to ensure each fold maintains the minority class balance and provides unbiased performance estimates)

3.2 Hyperparameter Space (6-dimensional)

Parameter	Type	Range	Scale	Rationale
learning_rate	continuous	[0.005, 0.3]	log-uniform	Controls step size; log scale for better coverage
num_leaves	integer	[20, 150]	uniform	Tree complexity; linear range appropriate
max_depth	integer	[3, 12]	uniform	Tree height; prevents overly deep trees
subsample	continuous	[0.5, 1.0]	uniform	Row subsampling; reduces overfitting
reg_lambda	continuous	[1e-4, 10]	log-uniform	L2 regularization; log scale for wide range
scale_pos_weight	continuous	[10, 80]	uniform	Class weight penalty; addresses imbalance explicitly

Note: The implemented notebook optimizes a 6-dimensional BO search space, which is consistent with the current experimental setup.

3.3 Baseline Methods

1. **Default LightGBM:** No hyperparameter tuning (scikit-learn default parameters). Establishes the floor performance.
2. **Coarse Grid Search:** 3×3×3 grid over:
 - learning_rate ∈ {0.01, 0.1, 0.2}
 - num_leaves ∈ {30, 80, 150}

- `max_depth` $\in \{5, 8, 12\}$
 - All other hyperparameters at defaults
 - Total evaluations: 27
3. **Random Search:** 50 random samples from the full 6-dimensional hyperparameter space (same budget as BO for fair comparison).
 4. **Optional Bayesian baseline:** A simple open-source tuner such as Optuna or Hyperopt can be added in future work to benchmark BO against a practical industry tool.

All implemented methods use:

- **Primary Metric:** Average Precision (area under precision-recall curve). Chosen because it is more sensitive to minority class performance than AUC-ROC, and appropriate for imbalanced classification
- **Validation:** Stratified 5-fold cross-validation

3.4 Bayesian Optimization Experiments

BO-EI: Bayesian Optimization with Expected Improvement

- Acquisition: EI with $\xi = 0.01$ (minimal exploration bonus for exploitation-focused search)
- GP kernel: Matern with $\nu = 2.5$
- Budget: 50 function evaluations

BO-LCB: Bayesian Optimization with Lower Confidence Bound (a minimization variant of UCB)

- Variants: $\beta \in \{0.5, 1.5, 3.0\}$
- GP kernel: Matern with $\nu = 2.5$
- Budget: 50 function evaluations per variant
- Purpose: Demonstrate empirically how β controls exploration/exploitation

3.5 Production Relevance and Deployment Perspective

This project is designed with practical deployment considerations in mind. A data science team would need to evaluate whether the tuned model is stable, auditable, and maintainable in a production fraud detection pipeline.

Key operational considerations include:

- **Monitoring data drift:** Fraud patterns evolve, so the model and its tuned hyperparameters should be retrained periodically.

- **Alert cost tradeoff:** A high false positive rate can damage customer experience, while false negatives can cause financial loss.
 - **Retraining cadence:** The pipeline should support scheduled retraining using new transaction data and updated CV folds.
 - **Explainability and auditability:** Feature importance and model behavior should be documented for stakeholders and potential audit review.
 - **Compute budget:** 50 BO evaluations are chosen to balance the cost of expensive CV-based tuning with the need for better-than-random performance.
-

4. Simulation Study: Proof-of-Concept

Before applying BO to the complex fraud detection task, we validate the algorithm's behavior on synthetic test functions with **known global optima**. This isolates BO's statistical properties from real-data noise and allows us to verify convergence.

4.1 Branin Function (2D, 3 Global Optima)

Definition:

$$f(x_1, x_2) = a(x_2 - bx_1^2 + cx_1 - r)^2 + s(1 - t) \cos(x_1) + s$$

Parameters: $a = 1, b = \frac{5.1}{4\pi^2}, c = \frac{5}{\pi}, r = 6, s = 10, t = \frac{1}{8\pi}$

Properties:

- **Domain:** $x_1 \in [-5, 10], x_2 \in [0, 15]$
- **Global minimum:** $f^* \approx 0.397$ (achieved at 3 equivalent locations)
- **Challenge:** Multiple local optima; non-convex landscape

Expected Behavior: BO should converge to one of the three global minima, significantly outperforming random search.

4.2 Hartmann-3 Function (3D, 1 Global + 4 Local Optima)

Properties:

- **Domain:** $x_i \in [0, 1]$ for $i = 1, 2, 3$
- **Global minimum:** $f^* \approx -3.86$
- **Challenge:** 4 local optima create misleading attraction zones

- **Relevance:** More similar to fraud hyperparameter landscape in dimensionality and complexity

Expected Behavior: BO's uncertainty quantification should help escape local optima, while random search may get trapped.

Purpose of Simulation Study:

1. Validate that GP posterior converges to the true optimum
 2. Show how acquisition functions guide search trajectories
 3. Compare convergence speed: BO-EI vs BO-UCB vs random search
 4. Visualize posterior mean and variance evolution across iterations
 5. Provide confidence that BO implementation is correct before running expensive real experiments
-

5. Analyses and Visualizations

5.1 Convergence Comparison

The notebook includes convergence plots that compare Bayesian optimization, CMA-ES, random search, grid search, and default LightGBM.

- **x-axis:** Cumulative function evaluations
- **y-axis:** Best validation Average Precision observed so far
- **Lines:** BO-EI, BO-LCB variants, CMA-ES, random search, default model
- **Purpose:** Demonstrate how efficiently each method discovers high-performing hyperparameters, especially under a tight evaluation budget.

5.2 GP Posterior and Acquisition Interpretation

The analysis visualizes the GP surrogate and acquisition behavior for the synthetic benchmarks:

- **GP posterior mean surface** after several BO iterations
- **EI acquisition surface** slices on Hartmann-3
- **Interpretation:** Show how the surrogate learns the objective landscape and selects candidates that balance exploration and exploitation.

5.3 Fraud Tuning Workflow

For the IEEE-CIS fraud detection application, the notebook documents:

- data loading and preprocessing from `data/train_transaction.csv` and `data/train_identity.csv`
- missing-value handling, label encoding, and stratified 5-fold CV
- LightGBM model evaluation on Average Precision
- hyperparameter search across default, grid search, random search, CMA-ES, and BO variants

5.4 Hyperparameter Importance

The notebook fits an ARD-enabled GP to the BO evaluations and ranks hyperparameters by `lengthscale`.

- **x-axis:** Hyperparameter names
- **y-axis:** Relative importance from normalized inverse lengthscales
- **Insight:** Identify which tuning parameters most strongly influence fraud detection performance.

5.5 Final Method Comparison

The notebook summarizes the tuning comparison in a concise table and focuses on the best method identified by the analysis.

- **Metrics:** Mean Average Precision from stratified CV
- **Methods:** Default LightGBM, grid search, random search, CMA-ES, BO-EI, BO-LCB variants
- **Decision point:** Prefer the method that delivers the highest AP with the smallest evaluation budget.

5.6 Business Impact

Translate the technical findings into operational value:

- a tuned fraud model should reduce missed fraud while controlling alert volume
 - `scale_pos_weight` tuning is especially important in imbalanced classification
 - Average Precision is the preferred metric because it is sensitive to the minority class performance that matters in fraud detection
-

6. Project Development Summary

This project progressed from synthetic benchmark validation to real-world application. The notebook implementation in `analysis/STAT238_Final_Project_v7.ipynb` executes the full Bayesian optimization pipeline and compares it against practical baselines. The report is now framed as a completed case study rather than a planning document.

7. Why This Project is Meaningful

For STAT 238 (Bayesian Statistics):

- **Deep Theoretical Engagement:** Derive GP posterior from first principles, implement from scratch, then compare to scikit-learn
- **Acquisition Functions:** Rigorous derivation of EI with connection to expected utility in Bayesian decision theory
- **Empirical Trade-offs:** Hands-on exploration of explore/exploit balance through systematic β variation
- **Posterior Interpretation:** Visualize how uncertainty and mean evolve as we gather evidence about the objective function

For Applied Data Science:

- **Real-world Challenge:** Fraud detection is a high-stakes, imbalanced classification problem where hyperparameter tuning directly impacts financial risk
- **Metric Choice Matters:** Show explicitly that Average Precision is the preferred metric for this imbalanced fraud detection task, while AUC-ROC can be included later for a complementary ranking perspective
- **Cost-Aware Optimization:** Demonstrate BO's advantage in budget-constrained settings (50 evaluations vs 1000+ for grid search)
- **Reproducible Methodology:** All code, hyperparameter ranges, and evaluation protocols documented for future replication

For the STAT 238 Portfolio:

- **Comprehensive Scope:** Spans simulation, theory, methodology, and application
- **Presentation-Ready:** Visualizations suitable for a 20 to 25 minute recorded talk

- **Grading Alignment:** Explicitly addresses all required theoretical components (GP posterior, EI, UCB)
-

8. Simulation Results Summary

Simulation Study: Branin Function

Purpose: Validate BO convergence on a 2D test function with 3 known global optima at $f^* \approx 0.3979$.

Methods Compared:

- **BO-EI:** Bayesian Optimization with Expected Improvement
- **BO-LCB ($\beta=1.5$):** Bayesian Optimization with Lower Confidence Bound
- **BO-LCB ($\beta=0.5$):** More exploratory LCB
- **BO-LCB ($\beta=3.0$):** More conservative LCB
- **CMA-ES**
- **Random Search**

Results (mean across 3 seeds):

- **BO-LCB ($\beta=0.5$):** mean best value 0.3981, best observed 0.3979
- **BO-LCB ($\beta=1.5$):** mean best value 0.4051, best observed 0.3981
- **BO-EI:** mean best value 0.5656, best observed 0.4146
- **CMA-ES:** mean best value 0.8316, best observed 0.4014
- **Random Search:** mean best value 1.4526, best observed 0.4094

Key Findings:

1. **BO variants are sample-efficient:** LCB and EI locate near-optimal solutions with fewer evaluations than random search.
2. **Acquisition choice matters:** LCB with a lower β was the most reliable on Branin, reaching the known minimum consistently.
3. **Random search is less stable:** It requires substantially more samples to match BO's performance.

Conclusion: The notebook's BO implementation is validated on benchmark functions and supports the transition to the fraud application.

Simulation Study: Hartmann-3 Function

Purpose: Evaluate BO on a 3D multimodal function with strong local optima.

Results (mean across 3 seeds):

- **BO-LCB ($\beta=1.5$):** mean best value -3.8490, best observed -3.8526
- **BO-EI:** mean best value -3.8268, best observed -3.8614
- **CMA-ES:** mean best value -3.5064, best observed -3.5739
- **Random Search:** mean best value -3.2314, best observed -3.4560

Key Findings:

1. **BO is effective in higher dimensions:** Both BO-EI and BO-LCB reached values much closer to the true optimum than CMA-ES and random search.
2. **BO-EI remains competitive:** It achieved the best single run on Hartmann-3, demonstrating the acquisition function can adapt to multimodal landscapes.
3. **Benchmarking helps validate methodology:** These synthetic results justify applying BO to fraud detection.

Conclusion: The notebook's BO implementation is validated and ready for application to the fraud detection hyperparameter search.

9. Business Impact and Deployment Perspective

A data science team would evaluate this work not only on model accuracy, but also on whether the final solution is operationally viable.

Deployment Considerations

- **Retraining schedule:** Use sliding windows or periodic refresh based on new transaction volumes and evolving fraud patterns.
- **Model monitoring:** Track key metrics such as AP, false positive rate, and prediction drift over time.
- **Alert management:** Translate model outputs into risk scores and threshold policies aligned with operational capacity.
- **Auditability:** Maintain records of feature handling, hyperparameter settings, and CV methodology to support reproducibility.

Career-Relevant Value

- Shows the ability to connect statistical modeling to business decision-making.
- Demonstrates an understanding of real-world constraints: compute cost, imbalance, and regulatory/audit concerns.
- Positions the work as a practical fraud detection study rather than a purely academic benchmark.

Final Recommendation

- Use BO to tune the fraud model under a strict evaluation budget.
 - Report both AP-focused and AUC-focused results, with a recommendation based on stakeholder priorities.
 - Emphasize the production-ready aspects: monitoring, retraining, and alert cost tradeoffs.
-

Code Organization

The main project notebook (`analysis/STAT238_Final_Project_v7.ipynb`) contains the following sections:

1. **Imports and Setup:** NumPy, pandas, scikit-learn GP, matplotlib, LightGBM
 2. **Test Functions:** Branin and Hartmann-3 implementations
 3. **Gaussian Process Surrogate:** GP initialization with Matern kernel
 4. **Bayesian Optimizer Class:** Core implementation of EI and LCB acquisition
 5. **Simulation Study:** BO on Branin and Hartmann-3, convergence visualizations
 6. **Fraud Detection Application:** data loading, preprocessing, LightGBM CV, and hyperparameter tuning
 7. **Benchmarking:** default LightGBM, grid search, random search, CMA-ES, BO variants
 8. **Hyperparameter Importance:** GP ARD analysis to rank influential parameters
 9. **Result Interpretation:** model comparison, practical insights, and deployment considerations
-

Demonstration Results and Recommendations

- **Goal:** demonstrate a reproducible, efficient tuning pipeline with a quick sanity run to validate the code, CV procedure, and search orchestration on a small compute budget.
- **Representative quick-sanity outputs (demo):** CMA-ES AP ≈ 0.4381 ; BO-EI candidate (illustrative): `learning_rate=0.3`, `num_leaves=86`, `max_depth=12`, `subsample=1.0`, `reg_lambda=10.0`, `scale_pos_weight=10.0`.
- **Key caveat:** these values come from a deliberately small budget (reduced sample fraction, fewer CV folds, BO iterations ~ 20) and should not be reported as final performance.
- **Recommendations:**
 - Include a scripted quick-sanity demo (already enabled via `tmp_fraud_run.py` SANITY_RUN) to show reproducibility and runtime control.
 - In any claims about model performance, present results from a full production tuning stage: full dataset, stratified 5-fold CV, BO budget ≥ 50 evaluations, and `n_estimators` ≥ 200 with early stopping; report mean AP $\pm$ std.
 - Add a short reproducibility note listing commands to run both the quick demo and the full tuning, and report approximate wall-time for each stage.
 - Emphasize methodology understanding (GP surrogate, acquisition functions, BO budget trade-offs) in presentation slides or README rather than raw metric comparisons from quick runs.

References

- Brochu, E., Cora, V. M., & de Freitas, N. (2010). A tutorial on Bayesian optimization. arXiv preprint arXiv:1012.2599.
- Mockus, J. (1974). On Bayesian methods for seeking the extremum. Optimization Techniques IFIP Technical Conference.
- Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N. (2016). Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(2), 148-175.
- LightGBM Documentation: <https://lightgbm.readthedocs.io/>
- Scikit-optimize Documentation: <https://scikit-optimize.github.io/>